

Автомати

Обмеження часу: 2 секунди

Обмеження пам'яті: 1024 MiB

Є поле, що складається з N клітинок, розташованих у ряд і пронумерованих зліва направо від 0 до $N - 1$. Кожна клітинка має унікальну висоту: висота клітинки i дорівнює p_i , а послідовність $p_0, p_1, \dots, p_{N-1}$ є перестановкою чисел $0, 1, \dots, N - 1$.

Дві клітинки з **індексами** i та j називаються *близькими* тоді й лише тоді, коли $|i - j| \leq 1$. Зокрема, кожна клітинка є близькою до самої себе.

На полі стоїть робот, якого спочатку розміщено в деякій клітинці. Робот приймає команди одного з таких двох типів:

- **MAX**: серед усіх клітинок, близьких до поточної клітинки робота, наступною позицією робота стане єдина клітинка з **максимальною** висотою;
- **MIN**: серед усіх клітинок, близьких до поточної клітинки робота, наступною позицією робота стане єдина клітинка з **мінімальною** висотою.

Оскільки клітинка є близькою до самої себе, після виконання команди робот може залишитися в тій самій клітинці.

Програма — це скінченна послідовність команд, кожна з яких є або **MAX**, або **MIN**. Якщо робот починає в клітинці X і виконує програму S , то він завершує рух у деякій однозначно визначеній клітинці, яку позначимо через $\text{result}(S, X)$.

Вам буде задано Q запитів, кожен із яких містить деяку множину початкових клітинок розміру K_i ($0 \leq i < Q$). Точніше, в i -му запиті буде задано клітинки $x_0, x_1, \dots, x_{K_i-1}$; робота буде розміщено в одній із них, але заздалегідь невідомо, у якій саме. Для кожного запиту потрібно визначити, чи існує програма, яка перемістить робота в ту саму кінцеву клітинку незалежно від того, яку початкову позицію з масиву x буде використано.

Формально, потрібно визначити, чи існує програма S така, що:

$$\text{result}(S, x_0) = \text{result}(S, x_1) = \dots = \text{result}(S, x_{K_i-1}).$$

Перестановка p є **однаковою** для всіх запитів.

Зверніть увагу, що вам не потрібно будувати таку програму — потрібно лише повідомити, чи існує вона.

Деталі реалізації

Перша функція, яку ви повинні реалізувати:

```
void initialize(std::vector<int> p)
```

- p : перестановка чисел від 0 до $N - 1$.

Друга функція, яку ви повинні реалізувати:

```
bool exists_program(std::vector<int> x)
```

- x : масив, що містить задані в запиті клітинки у строго зростаючому порядку.

Функцію `initialize` буде викликано рівно один раз, перед усіма викликами функції `exists_program`.

Функцію `exists_program` буде викликано Q разів, по одному разу для кожного запиту. Вона повинна повернути `true`, якщо існує програма, після виконання якої робот опиниться в тій самій клітинці незалежно від того, у якій із заданих клітинок він почав рух, і `false` інакше.

Обмеження

- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq Q \leq 5 \cdot 10^5$
- $p_0, p_1, \dots, p_{N-1}$ є перестановкою чисел $0, 1, \dots, N - 1$
- $2 \leq K_i \leq N$
- $0 \leq x_0 < x_1 < \dots < x_{K_i-1} < N$ для кожного запиту
- Сума значень K_i за всіма Q запитами не перевищує 10^6 .

Приклади

Приклад вхідних даних для градера 1	Приклад вихідних даних градера 1
3 3 0 2 1 2 0 2 3 0 1 2 2 1 2	111

Приклад вхідних даних для градера 2	Приклад вихідних даних градера 2
7 3 0 4 2 1 3 5 6 3 0 1 3 3 1 3 4 2 3 6	001

Пояснення першого прикладу

У цьому прикладі $p = [0, 2, 1]$. У другому запиті робот може почати рух із клітинки 0, клітинки 1 або клітинки 2. Розглянемо програму `[MAX]`.

- Якщо робот починає в клітинці 0: близькими до 0 є клітинки з індексами 0 та 1. Коли робот виконує команду `MAX`, оскільки $p_0 < p_1$, він переходить у клітинку 1.
- Якщо робот починає в клітинці 1: близькими до 1 є клітинки з індексами 0, 1 та 2. Коли робот виконує команду `MAX`, оскільки $p_0 < p_1$ та $p_1 > p_2$, він залишається в клітинці 1.
- Якщо робот починає в клітинці 2: близькими до 2 є клітинки з індексами 1 та 2. Коли робот виконує команду `MAX`, оскільки $p_1 > p_2$, він переходить у клітинку 1.

Отже, існує програма, після виконання якої робот із кожної з трьох початкових клітинок опиняється в клітинці 1, тому відповідь на цей запит — `true`. Також підходять інші програми, наприклад `[MIN, MAX]`, `[MIN, MIN, MAX, MIN, MAX, MIN]` та `[MAX, MIN, MAX]`.

Пояснення другого прикладу

У цьому прикладі $p = [0, 4, 2, 1, 3, 5, 6]$.

Для перших двох запитів можна довести, що не існує програми, після виконання якої робот опиниться в тій самій клітинці з кожної заданої початкової клітинки, тому відповідь на обидва запити — `false`.

Розглянемо останній запит, у якому робот може почати рух із клітинки 3 або клітинки 6, а також програму `[MAX, MAX, MAX]`.

- Якщо робот починає в клітинці 3: близькими до 3 є клітинки з індексами 2, 3 та 4. Оскільки $p_3 < p_4$ та $p_2 < p_4$, робот переходить у клітинку 4. Аналогічно виконавши наступні дві команди, він опиняється в клітинці 6.
- Якщо робот починає в клітинці 6, він протягом усього виконання програми залишається в клітинці 6 і завершує рух у ній.

Отже, відповідь на цей запит — `true`. Програма `[MIN, MAX, MAX, MAX]` також підходить. Однак інші програми можуть не підходити. Наприклад, якщо програма має вигляд `[MAX, MAX]`, то робот завершує рух у клітинці 5, якщо він почав у клітинці 3, але завершує рух у клітинці 6, якщо він почав у клітинці 6.

Підзадачі

Підзадача	Бали	N	Q	K_i	Додаткові обмеження
0	0	–	–	–	Приклади.
1	3	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$= 2$	Якщо відповідь на запит позитивна, то існує коректна програма, що складається рівно з 1 команди.
2	7				Якщо відповідь на запит позитивна, то існує коректна програма, що складається не більш ніж із 5 команд.
3	9	≤ 100	≤ 500	$= 2$	–
4	17	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$= 2$	–
5	7	$\leq 2 \cdot 10^5$			$x_1 = x_0 + 1$ для кожного запиту.
6	8	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$\leq N$	–
7	13	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$\leq N$	Перестановка спочатку зростає, потім спадає, а потім знову зростає. Формально, існують два цілі числа a, b такі, що $0 < a < b < N - 1$ та $p_0 < p_1 < \dots < p_a > p_{a+1} > \dots > p_b < p_{b+1} < \dots < p_{N-1}$.
8	13				$p_0 < p_1 > p_2 < p_3 > \dots < p_{N-1}$, якщо N парне, або $p_0 < p_1 > p_2 < p_3 > \dots > p_{N-1}$, якщо N непарне.
9	23				–

Приклад градера

Формат вхідних даних:

- рядок 1: два цілих числа — значення N та Q ;
- рядок 2: N цілих чисел $p_0, p_1, \dots, p_{N-1}$, де p_i — висота i -ї клітинки;
- рядок $3 + i$: клітинки, задані в i -му запиті, — ціле число K_i , після якого в тому самому рядку задано K_i цілих чисел $x_0, x_1, \dots, x_{K_i-1}$ — можливі початкові клітинки робота.

Формат вихідних даних:

- рядок 1: бінарний рядок довжини Q , у якому i -й символ дорівнює 1, якщо відповідь на i -й запит — true, і 0 інакше.